MCU Explorer – Mikrocontroller verstehen

Konzept der Linie MCU Explorer im DC Learning Ecosystem (Arduino-Uno-kompatible Plattform, ATmega328P)

Dokument	Konzept
Version	0.1
Status	ENTWURF
Autor	Daniel Zeimentz · DC Software Engineering
Datum	15.10.2026
Bezug	DC Learning Ecosystem – Gesamtkonzept v1.1, Konzept und Marktanalyse v1.0, DC Learning Design Standard v1.0; Konzept Volta v3.0; Konzept Zuse v0.3; ADR-0002 (Qt-freier Kern), ADR-0006 (keine Datenerhebung)
Zielplattformen	Windows, Linux, macOS (zuerst); Android-Tablet und iPadOS nach Machbarkeitsnachweis

Inhalt

1. Zusammenfassung **3**

2. Ausgangslage **3**

2.1 Stellung im Ecosystem . . . 3

2.2 Didaktischer Bruch, den der MCU Explorer schließt . . . 3

2.3 Markt und Wettbewerb . . . 4

3. Ziele und Nicht-Ziele **4**

3.1 Ziele . . . 4

3.2 Nicht-Ziele . . . 4

4. Abgrenzung zu Zuse 3 **5**

4.1 Arbeitsteilung . . . 5

4.2 Gemeinsam genutzt (Darstellung, nicht Inhalt) . . . 5

4.3 Übergänge . . . 5

4.4 Folgen für das Zuse-Konzept . . . 5

5. Lösungskonzept **5**

5.1 Arbeitsfläche . . . 5

5.2 Programmieren . . . 6

5.3 Simulation . . . 6

5.4 Übersetzung (Toolchain) . . . 7

5.5 Brücke zur echten Hardware (optional, später) . . . 7

6. Lernpfade **7**

6.1 MCU Explorer 1 (13–16 Jahre, Sek. I/II, Maker) . . . 7

6.2 MCU Explorer 2 (15–18 Jahre, Oberstufe, Berufsausbildung) . . . 8

7. Technische Architektur **8**

8. Rechte und Marken **8**

9. Anforderungen (MCU Explorer 1) **9**

10. Risiken und Maßnahmen **9**

11. Umsetzungsplan **10**

12. Offene Entscheidungen **10**

13. Fazit und Empfehlung **10**

Anhang A: Begriffe **11**

Anhang B: Referenzen **11**

1. Zusammenfassung

Der MCU Explorer ist das dritte Glied der Kernlinie **Volta → Zuse → MCU Explorer**. Lernende schreiben echte Programme in C/C++ für einen Arduino-Uno-kompatiblen Mikrocontroller (ATmega328P) und sehen, was ihr Code elektrisch bewirkt: Die Firmware läuft in einem befehlsgenauen Emulator des Controllers, seine Anschlüsse sind mit dem Simulationskern von Volta gekoppelt. Eine LED ohne Vorwiderstand überlastet den Pin, ein Taster ohne Pull-up liefert unbestimmte Pegel, ein Motor braucht Transistor und Freilaufdiode – so wie in der Wirklichkeit, aber ohne Risiko.

Wettbewerber wie Wokwi und Tinkercad Circuits simulieren den Arduino Uno bereits gut. Ein weiterer Uno-Simulator wäre kaum differenziert. Das Produktversprechen ist deshalb die **Verbindung von echter Firmware, ehrlicher Elektrik und strukturiertem Unterricht**: Grenzwertverletzungen, Messgeräte, Fehlersuche, Lernpfad mit Forscherfragen, offline und ohne Konto, Bedienung wie Volta.

Gegenüber **Zuse 3** gilt eine klare Arbeitsteilung: Zuse zeigt, **wie** ein Prozessor innen aufgebaut ist (aus Gattern, Registern und einem eigenen Befehlssatz). Der MCU Explorer zeigt, **wie man einen echten Mikrocontroller benutzt** – mit C/C++, Peripherie-Registern und elektrischer Umgebung. Gemeinsam genutzt werden Darstellungen (Register als LED-Reihe, Zahlenleiste, Logikanalysator), nicht Lerninhalte.

Empfehlung: Machbarkeitsnachweis »C/C++ → AVR-Maschinencode → GPIO → simulierte LED« auf dem Desktop (Ecosystem-Priorität P2), parallel die Abgrenzung zu Zuse 3 festschreiben. Ausbau erst nach Validierung von Volta 1 und Zuse 1.

2. Ausgangslage

2.1 Stellung im Ecosystem

Linie	Produkte	Rolle des MCU Explorers
Kernlinie	Volta 1–3 → Zuse 1–3 → **MCU Explorer**	Endpunkt: programmierbare Hardware, die eine elektrische Schaltung steuert
IT-Linie	PC Explorer → OS → Network → Internet & Cloud	Firmware-Verständnis als Grundlage für Treiber, Betriebssysteme und vernetzte Geräte
Suite	gemeinsame Lernumgebung	Projekt »Intelligente Wetterstation«: Sensor (Volta) → Datenverarbeitung (MCU) → Übertragung (Network) → Darstellung (Cloud)

2.2 Didaktischer Bruch, den der MCU Explorer schließt

Im Maker- und Schulalltag stehen zwei Welten nebeneinander: der Aufbau auf dem Steckbrett, der an Wackelkontakten, Toleranzen und zerstörten Bauteilen scheitert, und die Programmierung mit Arduino-Funktionen (`digitalWrite`, `analogRead`), die den Chip als Blackbox behandelt. Der MCU Explorer verbindet beide: Jede Codezeile wirkt sichtbar auf Register, Pins, Spannungen und Ströme.

2.3 Markt und Wettbewerb

Anbieter	Stärke	Lücke, die der MCU Explorer füllt
Wokwi	viele Controller (ATmega328P, ESP32, RP2040), Logikanalysator, Debugging, im Browser	Online-Werkzeug; Elektrik idealisiert; kein Lernpfad mit Forscherfragen
Tinkercad Circuits	einfacher Einstieg, Blöcke und Code, Schaltungen	Konto und Netz nötig; begrenzte Elektrik, keine Grenzwertfolgen
Reale Arduino-Kästen	echte Hardware, haptisch	Bauteilverschleiß, Fehlersuche schwer, Klassensätze teuer

Die Marktchance ist laut Ecosystem-Analyse »hoch (Hypothese)«; Kaufbereitschaft ist mit Lehrkräften, Ausbildungsbetrieben und Verlagen zu prüfen.

3. Ziele und Nicht-Ziele

3.1 Ziele

- **Echter Code:** Programme in C/C++ nach Arduino-Art (setup/loop, Bibliotheksfunktionen) und später direkt über Register.
- **Echte Ausführung:** Die Firmware läuft als AVR-Maschinencode in einem befehlsgenauen Emulator des ATmega328P (16 MHz).
- **Ehrliche Elektrik:** Pins treiben und messen über den Volta-Kern; Grenzwerte (Strom je Pin, Gesamtstrom, Spannungen) haben Folgen.
- **Sichtbar machen:** Register als LED-Reihen, Zahlen binär/dezimal/hex, Signale im Logikanalysator und Oszilloskop, Einzelschritt im Code.
- **Lernpfad** mit Forscherfrage, Vermutung, Aufbau, Betrieb, Beobachtung, Erklärung (Design Standard).
- **Offline, ohne Konto**, mehrsprachig (DE, FR, EN, LU), Bedienung wie Volta.

3.2 Nicht-Ziele

- Keine Netzwerkfunktionen (WLAN, Bluetooth, Cloud): Das ist Sache von Network und Internet & Cloud Explorer und verletzt die Offline-Regel.
- Keine Nachbildung fremder Marken: kein Produktname mit »Arduino«, kein Nachbau von Logo oder Platinenbild; nur beschreibender Kompatibilitätshinweis nach den Markenrichtlinien.
- Kein Ersatz für eine vollständige Entwicklungsumgebung (keine beliebigen Bibliotheken aus dem Internet).
- Keine weiteren Controller-Familien in Version 1 (ESP32, RP2040 erst nach Validierung).
- Keine Nachbildung des Prozessorinneren aus Gattern – das leistet Zuse.

4. Abgrenzung zu Zuse 3

4.1 Arbeitsteilung

Thema	Zuse 3 – verstehen, wie es gebaut ist	MCU Explorer – benutzen am echten Chip
Unterbrechungen	Unterbrechungslogik aus Gattern und Registern bauen; Rücksprungadresse sichern	`attachInterrupt`, Interrupt-Vektoren, prellende Taster, Prioritäten
Zeitgeber	Zähler und Vergleicher selbst zusammensetzen	Timer konfigurieren, `millis()`, PWM-Modi, Frequenzen messen
Ein-/Ausgabe	Port als Speicheradresse bauen	GPIO, Pull-up, Pegelschwellen, Stromgrenzen, elektrische Lasten
Architektur	Von-Neumann und Harvard vergleichen, einfaches Fließband	ATmega328P als Harvard-Rechner: Flash, SRAM, EEPROM, PROGMEM
Sprache	Assembler des eigenen Kerns (Zuse-K)	C/C++; Assembler nur als Einblick (MCU Explorer 2)
Elektrik	keine (rein digital)	volle Rückwirkung über den Volta-Kern
Rechner	selbst gebauter Prozessorkern Zuse-K	fertiger, industrieller Mikrocontroller

4.2 Gemeinsam genutzt (Darstellung, nicht Inhalt)

- **Register als LED-Reihe** mit Bitnummern und Zahlenleiste (binär, dezimal, hex) – aus Zuse übernommen.
- **Logikanalysator** und Einzelschritt-Bedienung mit gleichen Begriffen und Symbolen (Design Standard).
- **Digital-Engine** `logic/` für rein digitale Zusatzbausteine (z. B. Schieberegister 74HC595-artig als eigenes Modell), Analog-Engine von Volta für die Schaltung.

4.3 Übergänge

- **Zuse 3 → MCU Explorer:** Die letzte Zuse-3-Stufe »Der echte Mikrocontroller« zeigt die Registerkarte des ATmega328P in der Zuse-Darstellung und verweist auf den MCU Explorer.
- **MCU Explorer → Zuse 3:** Die MCU-Stufe »Blick in den Chip« zeigt PORTB/DDRB/PINB als Zuse-Bedienfeld und verweist für den inneren Aufbau auf Zuse.
- **Volta → MCU Explorer:** Alle Bauteile, Messgeräte und Fehlerbilder von Volta stehen auf der Grundplatte bereit.

4.4 Folgen für das Zuse-Konzept

Zuse 3 behandelt Unterbrechungen und Zeitgeber nur auf Bauebene und verzichtet auf Programmierung in C/C++, Sensorik und Elektrik. Das Zuse-Konzept v0.3 (Abschnitte 4.1, 7.3) entspricht dem bereits und verweist auf dieses Dokument.

5. Lösungskonzept

5.1 Arbeitsfläche

Bedienung nach dem DC Learning Design Standard, Volta als Referenz:

- **Mitte:** Volta-Grundplatte mit Kontaktfeldern; die **MCU-Platine** ist ein großes Modul mit beschrifteten Anschlussfeldern (D0–D13, A0–A5, 5V, 3V3, GND, VIN, AREF, RESET). Eigene, neutrale Gestaltung (»Uno-kompatible Platine«), kein Nachbau des Originalbilds.

- **Links:** Bauteilfach aus dem Volta-Katalog (LED, Widerstand, Taster, Potentiometer, LDR, NTC, Summer, Lautsprecher, Transistor, Diode, Motor; später Servo, Schieberegister, Siebensegment).
- **Rechts:** Aufgabe mit Forscherfrage und Hilfe (Maskottchen oder neutrales Hilfesystem, offen).
- **Unten (umschaltbar):** Code-Editor, serieller Monitor, Registeransicht, Logikanalysator/Oszilloskop.
- **Symbol:** M als Leiterbahn auf grüner Platine, deren Enden in die Steckleiste führen; Band als LED-Balkenanzeige (Regel für die ganze Reihe: `docs/design/symbole/README.md`).
- **Kopfzeile:** Prüfen/Übersetzen, Hochladen, Start, Pause, Einzelschritt, Reset – gleiche Symbole wie Volta und Zuse.

5.2 Programmieren

Stufe	Form	Zweck
Einstieg	Arduino-Art: `setup()`/`loop()`, `pinMode`, `digitalWrite`, `analogRead`, `delay`, `Serial`	schneller Erfolg, bekannte Schreibweise
Vertiefung	Register direkt: `DDRB`, `PORTB`, `PINB`, Bitmasken, Timer- und ADC-Register	verstehen, was die Funktionen tun
Einblick	Disassembler-Ansicht des erzeugten Maschinencodes, Einzelschritt je Befehl	Brücke zu Zuse

- Editor mit Syntaxhervorhebung, Fehlermeldungen in verständlicher Sprache (mit Originalmeldung), Beispielen je Stufe.
- Einzelschritt je Quelltextzeile und je Maschinenbefehl; Haltepunkte; Variablen- und Registeransicht.

5.3 Simulation

MCU-Engine (neu, Qt-frei, C++17):

- Befehlsgenauer Emulator des AVR-Kerns (ATmega328P, 16 MHz, Taktzyklen je Befehl), Flash 32 KB, SRAM 2 KB, EEPROM 1 KB.
- Peripherie Version 1: GPIO (Ports B, C, D, Pull-ups), Timer 0/1/2 mit PWM, ADC 10 Bit, USART (serieller Monitor), externe und Pin-Change-Interrupts, EEPROM. Später: SPI, I²C (TWI), Watchdog, Schlafmodi.
- Eingabeformat: Intel HEX bzw. ELF aus der Übersetzung.

Kopplung an den Volta-Kern (Simulations-Koordinator):

Richtung	Modell
Ausgang	Pin als Spannungsquelle mit Innenwiderstand (High gegen VCC, Low gegen GND, nach Datenblattkennlinien); Strom wird gemessen
Eingang	Knotenspannung gegen Schwellen (VIL 0,3 · VCC, VIH 0,6 · VCC, Hysterese); dazwischen unbestimmt; interner Pull-up 20–50 kΩ
ADC	Knotenspannung gegen Referenz (AVCC/AREF) quantisiert, Abtastzeitpunkt aus der Emulation
Grenzwerte	40 mA je Pin, 200 mA gesamt, Spannungen außerhalb −0,5 … VCC + 0,5 V: Warnung, bei Dauer Zerstörung (»Nur in der App!«)
Versorgung	5 V über USB oder VIN über Spannungsregler (Volta-3-Modell); Brown-out bei Unterspannung

- **Zeitkopplung:** Der Emulator läuft in Zeitscheiben; jede Pinänderung ist ein Ereignis, an dem der Volta-Kern einen Teilschritt rechnet. Zwischen Ereignissen rechnet der Kern mit der üblichen Schrittweite. Grundlage ist der Leistungstest aus Volta 2 (lineare Schaltungen mit 20 µs auf dem Tablet mehrfach echtzeitfähig).

- **PWM** wird ereignisgenau gerechnet (keine Mittelwert-Abkürzung); Helligkeit und Klang entstehen aus der Simulation.

5.4 Übersetzung (Toolchain)

Weg	Plattform	Bewertung
A: avr-gcc mit avr-libc und Arduino-AVR-Core, mitgeliefert als eigenes Programm	Windows, Linux, macOS	echter C/C++-Ablauf; Größe (ca. 100 MB) und Lizenzpflichten (GPL mit Laufzeitausnahme, LGPL, BSD) prüfen
B: avr-gcc auf Android als mitgelieferte native Programmdatei	Android	technisch möglich, Größe und Store-Regeln prüfen
C: eigener Übersetzer für eine klar benannte Teilmenge (»Explorer-C«) direkt nach AVR-Maschinencode	alle, auch iPadOS	volle Kontrolle und klein; nur Teilmenge, muss ehrlich so benannt werden
D: Übersetzung am Desktop, Ergebnis (HEX) auf dem Tablet ausführen	alle	für Unterricht praktikabel, kein eigenständiges Tablet-Erlebnis

Vorschlag: Desktop zuerst mit Weg A. Für Tablets per Machbarkeitsnachweis zwischen B und C entscheiden; Weg D als Übergang. Der Emulator ist in allen Fällen derselbe.

5.5 Brücke zur echten Hardware (optional, später)

Auf dem Desktop kann ein Programm über USB auf ein reales Uno-kompatibles Board übertragen werden (avrdude). Kein Netz; Aufruf nur auf ausdrücklichen Wunsch. Entscheidung nach Version 1.

6. Lernpfade

6.1 MCU Explorer 1 (13–16 Jahre, Sek. I/II, Maker)

Stufe	Forscherfrage	Inhalt	Volta-Bezug
1	Wie bringe ich einen Chip zum Blinken?	setup/loop, `pinMode`, `digitalWrite`, `delay`; Hochladen	LED braucht Vorwiderstand – ohne überlastet der Pin
2	Wie merkt der Chip einen Tastendruck?	`digitalRead`, Pull-up, offene Eingänge, Prellen	Spannungsteiler, Taster
3	Wie läuft ein Licht von selbst?	Schleifen, Felder, Lauflicht	Gesamtstrom mehrerer LEDs
4	Wie dimmt man eine LED?	PWM mit `analogWrite`; Rechteck im Oszilloskop	Mittelwert, Tastverhältnis
5	Wie misst der Chip eine Spannung?	`analogRead`, Potentiometer, Lichtsensor	Messgerät und ADC-Wert vergleichen
6	Was denkt der Chip gerade?	serieller Monitor, Messwerte ausgeben	–
7	Wie spielt der Chip Musik?	`tone`, Summer und Lautsprecher (hörbar über Volta-2-Ton)	Lautsprecher, Strom
8	Wie schaltet ein schwacher Pin einen starken Motor?	Transistor als Schalter, Freilaufdiode	Volta-1-Transistor, Induktionsspitze
9	Wie wartet man, ohne stillzustehen?	`millis()` statt `delay`, mehrere Aufgaben	–
10	Wie reagiert der Chip sofort?	`attachInterrupt`, Prellen im Logikanalysator	–
11	Projekt	Ampel, Nachtlicht oder Messstation (offline)	eigener Aufbau

6.2 MCU Explorer 2 (15–18 Jahre, Oberstufe, Berufsausbildung)

Stufe	Thema	Inhalt
1	Blick in den Chip	Register DDRx/PORTx/PINx als Bedienfeld, Bitmasken
2	Timer selbst einstellen	Timer-Register, Vorteiler, Vergleich, PWM-Modi
3	ADC ohne Bibliothek	Referenz, Abtastzeit, Auflösung, Störungen
4	Interrupts im Detail	Vektoren, Flags, gemeinsame Variablen (`volatile`)
5	Serielle Schnittstelle auf Registerebene	Baudrate, Rahmen, Logikanalysator
6	Sparsam rechnen	Schlafmodi, Stromaufnahme, Watchdog
7	Bausteine am Bus	SPI und I²C mit simulierten Sensoren und Anzeigen
8	Was der Übersetzer daraus macht	Disassembler, Befehlszyklen, Optimierung
9	Projekt	eigenes Gerät mit Fehlersuche

7. Technische Architektur

Schicht	Inhalt
`mcu/` (neu)	AVR-Emulator und Peripherie, Qt-frei (C++17, wie `core/` und `logic/`)
Koordinator	Zeitscheiben, Ereignisse, Digital–Analog-Brücke zwischen `mcu/` und `core/`
`core/`	Volta-Simulationskern für die Schaltung (unverändert, erweitert um Pin-Modelle)
Toolchain-Dienst	Übersetzen, Fehlermeldungen aufbereiten, HEX/ELF laden (plattformabhängig)
App	Qt 6/QML, Design Standard; Grundplatte und Bauteile aus Volta, Register- und Analysatoransichten aus Zuse

- **Tests:** Befehlsgenauigkeit gegen veröffentlichte Befehlstabellen und Testprogramme; Peripherie gegen Referenzmessungen an realer Hardware (einmalig aufgenommene Signalverläufe).
- **Leistung:** Ziel mindestens Echtzeit (16 MHz) auf dem Referenztablet bei typischen Aufbauten; eigenes Messprogramm wie `volta_core_bench`.
- **Datenformate:** Projekte im gemeinsamen Ecosystem-Format (.dclearn): Aufbau, Quelltext, Einstellungen.

8. Rechte und Marken

- **Produktname** »MCU Explorer« (eigene Marke, Recherche offen). »Arduino« nur beschreibend nach den Arduino-Markenrichtlinien (z. B. »für Arduino-Uno-kompatible Programme«), kein Logo, kein Nachbau des Platinenbilds.
- **ATmega328P** (Microchip) wird beschreibend genannt; Registernamen und Befehlssatz aus dem Datenblatt sind technische Fakten.
- **Toolchain:** avr-gcc (GPL mit Laufzeitausnahme), avr-libc (BSD-artig), Arduino-AVR-Core (LGPL 2.1); Weitergabe nur mit erfüllten Lizenzpflichten (Quellangebot, Hinweise), juristische Prüfung vor Veröffentlichung.
- **Emulator:** Eigenentwicklung, keine Übernahme GPL-lizenzierter Simulatoren (z. B. simavr) in den geschlossenen Teil.

9. Anforderungen (MCU Explorer 1)

ID	Bereich	Anforderung	Priorität
M1	Ausführung	AVR-Kern befehlsgenau, 16 MHz, Flash/SRAM/EEPROM	MUSS
M2	Peripherie	GPIO, Timer/PWM, ADC, USART, externe und Pin-Change-Interrupts	MUSS
M3	Elektrik	Pins über den Volta-Kern; Grenzwerte mit Warnung und Zerstörung	MUSS
M4	Code	Editor, Übersetzen, verständliche Fehlermeldungen, Beispiele	MUSS
M5	Fehlersuche	Pause, Einzelschritt (Zeile, Befehl), Haltepunkte, Register- und Variablenansicht	MUSS
M6	Messen	Messgerät, Oszilloskop, Logikanalysator, serieller Monitor	MUSS
M7	Lernpfad	11 Stufen mit Forscherfrage, Hilfen, Musterlösung	MUSS
M8	Lehrkräfte	Präsentationsmodus, Aufgaben per QR-Code, Lehrerhandreichung	SOLL
M9	Hardware	Übertragung auf reales Board über USB (Desktop)	KANN
N1	Leistung	Echtzeit auf dem Referenztablet bei typischen Aufbauten	MUSS
N2	Datenschutz	offline, kein Konto, keine Datenerhebung	MUSS
N3	Bedienung	Design Standard; Touch ≥ 48 dp; Zustände nie nur über Farbe	MUSS
N4	Sprachen	DE, FR, EN, LU	MUSS

10. Risiken und Maßnahmen

Nr.	Risiko	Eintritt	Auswirkung	Maßnahme
R1	Emulator ungenau (Zeitverhalten, Peripherie)	MITTEL	HOCH	Testprogramme, Vergleich mit realer Hardware, schrittweiser Ausbau
R2	Toolchain auf Tablets (Größe, Store-Regeln, iPadOS)	HOCH	HOCH	Desktop zuerst; Machbarkeitsnachweis B/C; Weg D als Übergang
R3	Kopplung an den Volta-Kern zu langsam	MITTEL	MITTEL	ereignisgesteuerte Teilschritte; Leistungsmessung früh
R4	Kostenloser Wettbewerb (Wokwi, Tinkercad)	HOCH	MITTEL	Elektrik, Offline, Lernpfad und Unterrichtsmaterial als Unterschied; kein reiner Simulator
R5	Lizenz- und Markenfragen (Arduino, GPL-Toolchain)	MITTEL	HOCH	juristische Prüfung, eigener Emulator, beschreibende Kompatibilität
R6	Überschneidung mit Zuse 3	MITTEL	MITTEL	Arbeitsteilung nach Abschnitt 4, gemeinsame Darstellungen statt doppelter Inhalte
R7	Aufwand bindet Kapazität vor Volta 1.0	HOCH	HOCH	nur Machbarkeitsnachweis bis zur Verlagsrückmeldung

11. Umsetzungsplan

Phase	Inhalt	Grobschätzung	Ergebnis
M0	Machbarkeitsnachweis Desktop: AVR-Kern (Teilmenge), GPIO, avr-gcc, LED mit Vorwiderstand im Volta-Kern; Leistungsmessung	15–20 PT	»C/C++ → AVR → GPIO → LED« läuft reproduzierbar
M1	vollständiger AVR-Kern, Timer/PWM, ADC, USART, Interrupts; Koordinator	40–60 PT	Peripherie gegen Referenz geprüft
M2	App nach Design Standard: Grundplatte mit MCU-Platine, Editor, Monitor, Registeransicht; Stufen 1–6	30–40 PT	erste Testversion (Desktop)
M3	Stufen 7–11, Logikanalysator, Lehrkraftfunktionen; Tablet-Weg nach Machbarkeitsnachweis	40–60 PT	MCU Explorer 1 Testversion
M4	MCU Explorer 2	nach Version 1	–

12. Offene Entscheidungen

Entscheidung	Spätestens
Arbeitsteilung mit Zuse 3 bestätigen (Abschnitt 4)	vor Phase M0
Tablet-Weg der Übersetzung (B oder C)	Ende Phase M1
Maskottchen oder neutrales Hilfesystem	vor Phase M2
Altersstufe und Zielmärkte (Schule, Maker, Berufsausbildung) nach Verlagsrückmeldung	vor Phase M2
Übertragung auf reale Hardware (Abschnitt 5.5)	nach Version 1
Weitere Controller (ESP32 ohne Netzfunktionen, RP2040)	nach Version 1

13. Fazit und Empfehlung

Der MCU Explorer schließt die Kernlinie: Wer mit Volta Schaltungen versteht und mit Zuse weiß, wie ein Prozessor arbeitet, steuert jetzt eine echte Schaltung mit eigenem Code – und sieht, was der Code elektrisch bewirkt. Die Abgrenzung zu Zuse 3 ist eindeutig: Zuse baut den Prozessor, der MCU Explorer benutzt ihn. Technisch entscheidend sind der eigene AVR-Emulator, seine Kopplung an den Volta-Kern und der Übersetzungsweg auf Tablets; alle drei werden zuerst in einem kleinen Machbarkeitsnachweis geprüft.

Nächste Schritte: (1) Arbeitsteilung mit Zuse 3 bestätigen. (2) Machbarkeitsnachweis M0 nach der Verlagsrückmeldung zu Volta 1. (3) Markenrecherche »MCU Explorer« und Lizenzprüfung der Toolchain.

Anhang A: Begriffe

Begriff	Bedeutung
MCU	Mikrocontroller: Prozessor, Speicher und Peripherie auf einem Chip
Firmware	Programm, das im Mikrocontroller läuft
Emulator	Software, die einen Prozessor Befehl für Befehl nachbildet
GPIO	frei programmierbarer digitaler Ein-/Ausgang
PWM	Pulsweitenmodulation: Rechtecksignal mit einstellbarem Tastverhältnis
ADC	Analog-Digital-Wandler
USART	serielle Schnittstelle
Interrupt	Unterbrechung des Programms durch ein Ereignis
Koordinator	Teil, der Emulator und Volta-Kern zeitlich koppelt
Explorer-C	mögliche eigene Teilmenge von C/C++ für Tablets

Anhang B: Referenzen

- DC Learning Ecosystem – Gesamtkonzept v1.1; Konzept und Marktanalyse v1.0; DC Learning Design Standard v1.0 (Projektordner »Learning ECOSYSTEM«)
- Konzept Zuse v0.3 (`docs/Konzept_Zuse.md`), Konzept Volta v3.0, Volta-2-Planung (Leistungstest Audio)
- ATmega328P-Datenblatt (Microchip); Arduino-Markenrichtlinien für kompatible Produkte
- Wettbewerb: Wokwi (unterstützte Hardware), Tinkercad Circuits